

North Bengal International University



Lab Report: 2

Exp Name : A Program to Generate Histogram of a Grayscale Images

Subject : Digital Image Processing sessional

Subject Code : CSE 0613-4142

Submitted By:

Name : Md. Tanvir Rahman Pranto
Dept : CSE
Roll : 0712340012101005
Semester : 3nd Year, 6th semester.

Submitted To:

Name : Sakurah Ismail
Designation : Lecturer
Dept : CSE

Experiment Title: A Program to Generate Histogram of a Grayscale Images

Introduction: In digital image processing, a histogram is a graphical representation of the distribution of pixel intensities in an image. For grayscale images, it shows how frequently each intensity value (0–255) occurs. Histograms are widely used for image analysis, enhancement, and thresholding operations. This experiment demonstrates how to generate and visualize histograms for both grayscale and binary images using Python and OpenCV.

Code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import os

image_path = "/content/wallpaper.jpg"

if not os.path.exists(image_path):
    raise FileNotFoundError(f"Error: The file '{image_path}' does not exist. Please upload the image or provide the correct path.")

img_gray = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)

if img_gray is None:
    raise ValueError(f"Error: Could not load image from {image_path}. Check if the file is a valid image.")

print(f"Image '{image_path}' loaded successfully.")
print(f"Image Shape: {img_gray.shape}")

hist_gray = cv2.calcHist([img_gray], [0], None, [256], [0, 256])

plt.figure(figsize=(12, 6))

plt.subplot(1, 2, 1)
plt.imshow(img_gray, cmap='gray')
plt.title('Original Grayscale Image')
plt.axis('off') # Turn off axis for cleaner image display

plt.subplot(1, 2, 2)
plt.plot(hist_gray, color='black') # Plot histogram
```

```
plt.title('Histogram of Original Grayscale Image')
plt.xlabel('Pixel value')
plt.ylabel('Frequency')
plt.xlim([0, 256])
plt.grid(True, linestyle=':', alpha=0.7)

plt.tight_layout()
plt.savefig('output_grayscale_image_and_histogram.png') # Save plot
with a more descriptive name
plt.show()
```

```
threshold_value = 127 # Define the threshold value
```

```
# Apply binary thresholding
```

```
_, img_binary = cv2.threshold(img_gray, threshold_value, 255,
cv2.THRESH_BINARY)
```

```
hist_binary = cv2.calcHist([img_binary], [0], None, [256], [0, 256])
```

```
plt.figure(figsize=(12, 6))
```

```
plt.subplot(1, 2, 1)
```

```
plt.imshow(img_binary, cmap='gray') # Binary images are also
typically displayed with a 'gray' colormap
```

```
plt.title(f'Black and White Image (Threshold = {threshold_value})')
```

```
plt.axis('off')
```

```
plt.subplot(1, 2, 2)
```

```
plt.plot(hist_binary, color='blue') # Plot histogram for the binary
image
```

```
plt.title('Histogram of Black and White Image')
```

```
plt.xlabel('Pixel value')
```

```
plt.ylabel('Frequency')
```

```
plt.xlim([0, 256])
```

```
plt.grid(True, linestyle=':', alpha=0.7)
```

```
plt.tight_layout()
```

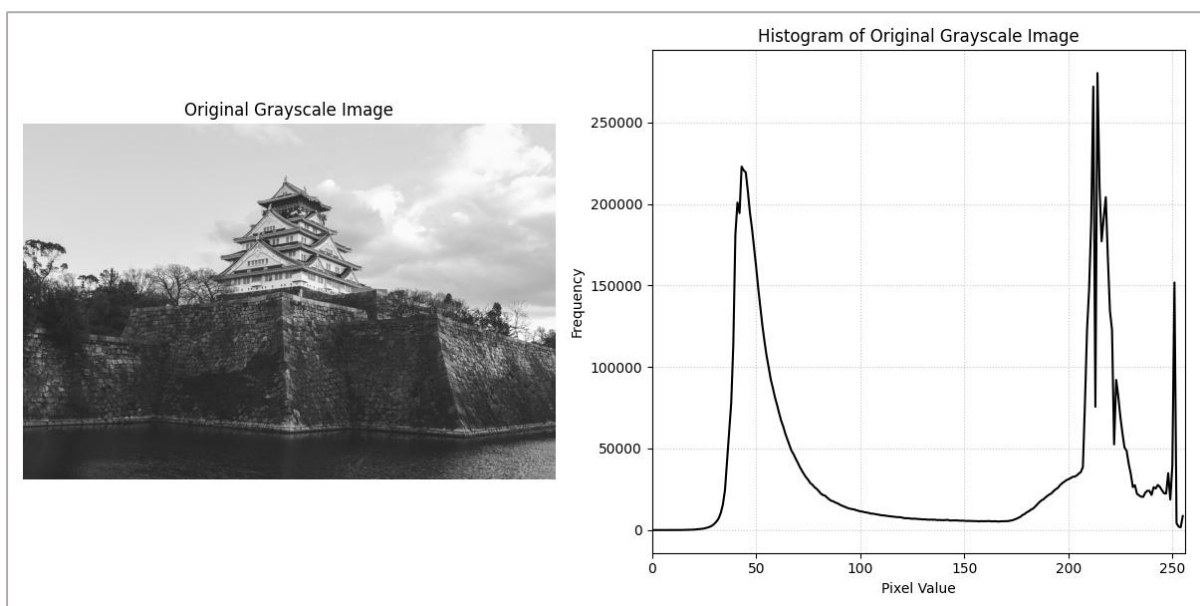
```
plt.savefig('output_binary_image_and_histogram.png') # Save plot
with a more descriptive name
```

```
plt.show()
```

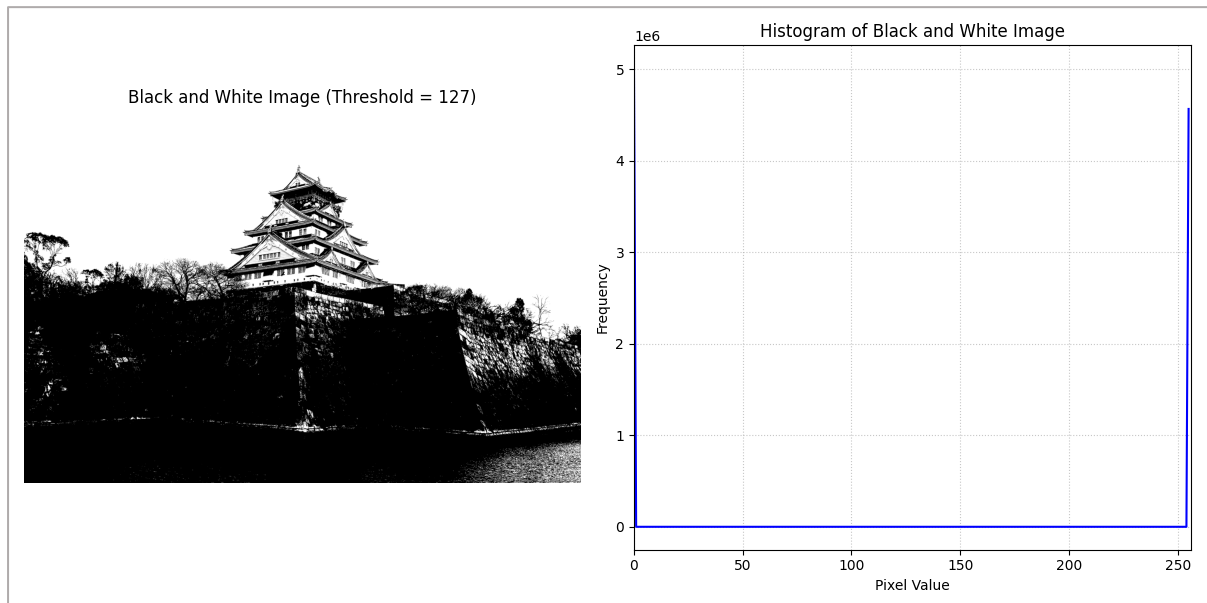
Input:



Output – 1 :



Output – 2 :



Discussion on the output: The program first loads a grayscale image and computes its histogram, which reveals the spread of pixel intensities. A balanced histogram indicates good contrast, while a narrow range suggests poor contrast. Next, binary thresholding is applied to convert the grayscale image into a black-and-white image. The histogram of the binary image shows peaks only at 0 and 255, representing black and white pixels. This highlights how thresholding simplifies intensity distribution and is useful in segmentation tasks. Overall, histograms provide valuable insights into image characteristics and guide preprocessing steps in computer vision.